

Part 1: Multiple Choice Questions (MCQs)

- 1) B
- 2) B
- 3) B
- 4) A
- 5) B
- 6) B
- 7) C
- 8) C
- 9) C

part 2: Short Answer / Problem-Based Questions

1. (5 Marks) Create a small dataset of 10 numerical values. Calculate the mean, median, variance, and standard deviation using Python.

```
import pandas as pd

data = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
df = pd.Series(data)

print(f"Mean: {df.mean()}")
print(f"Median: {df.median()}")
print(f"Variance: {df.var()}")
print(f"Standard Deviation: {df.std()}")
```

✓ 0.0s

```
Mean: 55.0
Median: 55.0
Variance: 916.6666666666666
Standard Deviation: 30.276503540974915
```

2. (5 Marks) Using Pandas, create a histogram with 4 bins for the following dataset:

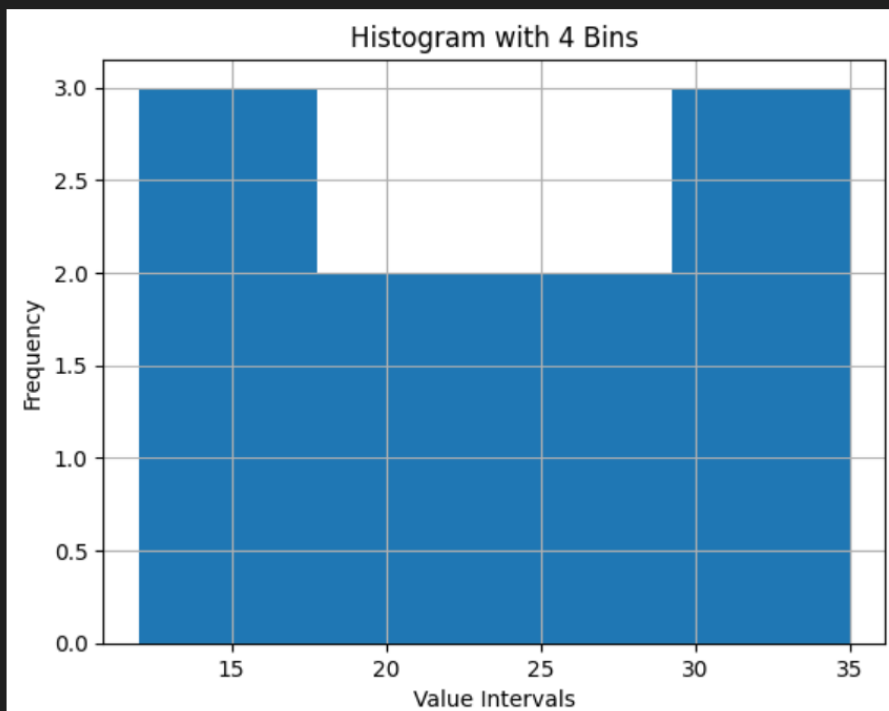
[12, 15, 17, 18, 22, 24, 28, 30, 32, 35]

```
import pandas as pd
import matplotlib.pyplot as plt

data = [12, 15, 17, 18, 22, 24, 28, 30, 32, 35]
df = pd.DataFrame(data, columns=['Values'])

df['Values'].hist(bins=4)
plt.title("Histogram with 4 Bins")
plt.xlabel("Value Intervals")
plt.ylabel("Frequency")
plt.show()
```

✓ 1.6s



3. (10 Marks) A dataset has values: [10, 12, 15, 18, 18, 20, 50].

a) Identify the outlier(s).

b) Calculate the mean and median.

c) Explain how the outlier affects the mean and median.

Answer :

a) Identify the outlier(s): The outlier is 50. It is significantly higher than the rest of the values in the group (10–20).

b) Calculate the mean and median:

a. Mean: $10+12+15+18+18+20+50 / 7 = 20.43$

b. Median: The middle value is 18.

c) Effect of the outlier: The outlier inflates the mean (pulling it up from what would be about 15.5 to 20.43). However, it has no effect on the median, which stays at 18. This shows the median is more "robust" to outliers.

4. (10 Marks) Explain the difference between variance and standard deviation. Include an example in Python showing both for a small dataset.

Answer:

Difference: Variance measures the average squared distance from the mean (expressed in squared units), while Standard Deviation is the square root of variance, bringing the measure back to the original units of the data, making it easier to interpret.

```
> import numpy as np

data = [2, 4, 6, 8, 10]
var = np.var(data)
std = np.std(data)

print(f"Variance: {var}")
print(f"Standard Deviation: {std}")
```

[5] ✓ 0.0s

```
... Variance: 8.0
Standard Deviation: 2.8284271247461903
```

5. (10 Marks) Write a custom Python function that takes a list of numbers and

returns:

o Mean

o Median

o Standard deviation

Demonstrate it on the dataset [5, 8, 10, 12, 15].

```
> ✓
  ✓ import numpy as np
    from statistics import median

  ✓ def calculate_stats(numbers):
    mean_val = np.mean(numbers)
    median_val = median(numbers)
    std_val = np.std(numbers)

    return mean_val, median_val, std_val

test_data = [5, 8, 10, 12, 15]
mean, med, std = calculate_stats(test_data)

print(f"Dataset: {test_data}")
print(f"Mean: {mean}")
print(f"Median: {med}")
print(f"Standard Deviation: {std:.2f}")
6] ✓ 0.0s

Dataset: [5, 8, 10, 12, 15]
Mean: 10.0
Median: 10
Standard Deviation: 3.41
```

6. (10 Marks) A dataset follows a normal distribution with mean = 100 and standard deviation = 10. Using the empirical rule, calculate the approximate percentage of values that lie between:

a) 90 and 110

b) 80 and 120

c) 70 and 130

```
mean = 100
std = 10

# a) 90 to 110 (1 standard deviation)
print(f"Percentage between 90-110: 68%")

# b) 80 to 120 (2 standard deviations)
print(f"Percentage between 80-120: 95%")

# c) 70 to 130 (3 standard deviations)
print(f"Percentage between 70-130: 99.7%")
```

✓ 0.0s

```
Percentage between 90-110: 68%
Percentage between 80-120: 95%
Percentage between 70-130: 99.7%
```

Part 3: Essay / Reflection Questions

1. (10 Marks) Explain how histograms, skewness, mean, median, variance, and standard deviation together provide a complete picture of a dataset.

Use an example dataset to illustrate your explanation.

Answer:

1. The Complete Picture of a Dataset

These statistical tools work together to describe three main aspects of data: Shape, Central Tendency, and Spread.

- **Shape (Histograms & Skewness):** A histogram provides a visual overview of how data is distributed. Skewness tells us if the data is balanced or leaning toward one side. For example, in a "salary" dataset, most people earn a mid-range salary, but a few millionaires create a "tail" on the right (positive skew).
- **Central Tendency (Mean & Median):** These tell us where the "center" of the data is. In a perfectly symmetrical dataset, they are equal. However, if there are outliers, the mean gets pulled away, while the median stays in the middle.
- **Spread (Variance & Standard Deviation):** These describe how much the data varies. A low standard deviation means the values are close to the mean, while a high one means the data is widely spread out.

Example Dataset: [5, 5, 5, 10, 100]

- **Mean (25):** Misleading because of the outlier (100).
- **Median (5):** More accurate for the typical value.
- **Standard Deviation (42.1):** High, indicating the data is very inconsistent.
- **Histogram:** Would show a huge gap between the small numbers and 100, indicating a clear positive skew.

2. (10 Marks) Reflect on the importance of Python and Pandas for statistical analysis. How do they help in handling real-world data compared to manual calculations? Include examples of at least two statistical functions or techniques you find most useful.

Answer:

2. Importance of Python and Pandas

Python and Pandas are essential for modern data analysis because they automate the "heavy lifting" that is impossible to do manually with large datasets.

Why they are better than manual calculations:

1. **Scalability:** Manually calculating the mean of 10 values is easy, but calculating it for 1,000,000 rows is impossible. Python does it in milliseconds.
2. **Reproducibility:** If the data changes, you just re-run the script. In manual work, you have to start from scratch.
3. **Data Cleaning:** Real-world data is "messy" (missing values, wrong formats). Pandas handles this automatically.

Two Most Useful Functions:

- **df.describe():** This is incredibly powerful because it provides the count, mean, standard deviation, and quartiles in a single line of code, giving an instant snapshot of the data.
- **df.fillna():** In real-world data, many entries are missing. This function allows us to handle those gaps (by replacing them with the mean or zero) so the analysis doesn't break.